

Machine Learning para Clasificar Usuarios de Aplicación Musical

Juan Sebastián Chitiva Guerrero

A01276239@tec.mx

Tecnológico de Monterrey

Querétaro, Querétaro, MX

Abstract

Se utiliza un dataset de información sobre una app llamada "Playlist Pro" con el propósito de analizar el uso y características principales de usuarios y así encontrar patrones para clasificar de manera binaria a los usuarios activos y los que han cancelado su suscripción. Este análisis contiene una auditoría de datos, la codificación de variables categóricas usando One Hot Encoding, estandarización de variables numéricas y análisis exploratorio de correlaciones. La clasificación se realiza inicialmente con un modelo de regresión logística sin uso de frameworks enfocados en Machine Learning y posteriormente se implementa un modelo con el framework de Scikit-learn con el propósito de encontrar un modelo más robusto y enfocado a este problema. Como métricas de evaluación se utiliza Accuracy, Recall de la clase positiva, matriz de confusión, AUC-ROC, PR-AUC, F1 score y precisión, enfocado principalmente en obtener un PR-AUC óptimo, manteniendo un Recall y precisión razonable. También se realiza un modelo de XGBoost el cual se ajusta tanto con hiperparámetros así como con su "threshold".

1 Introduction

La música es un rasgo relevante y muy importante para la vida de muchas personas alrededor del mundo. Es su forma de despejarse y de relajar la mente. Gracias a esto y con el avance de la tecnología, existen actualmente muchas aplicaciones para poder escuchar música. En este reporte, se busca analizar las características y formas de uso de los clientes para determinar que provoca a los usuarios cancelar la suscripción de la aplicación "Playlist Pro". Este análisis se realiza para poder implementar medidas que minimicen los casos de cancelación.

2 Comprensión de los datos

2.1 Información Inicial

Se ha hecho el análisis con un dataset que contiene 41,000 muestras y 20 columnas. Cada muestra es un usuario y las columnas determinan información característica sobre el usuario y el uso de la aplicación, métodos de pago, fecha de suscripción, entre otros. El dataset se puede encontrar en la siguiente página: https://www.kaggle.com/daliado98/music-streaming-customer-churn-dataset/data?select=sample_submission.csv

2.2 Objetivos Analíticos

Se busca construir un modelo de clasificación que permita determinar a los usuarios activos contra los clientes que han cancelado su suscripción. Este resultado será arrojado de forma binaria, en el cual el valor "1" determina al usuario que ha cancelado.

2.3 Definiciones y métricas clave

Definiciones:

- **Churned:** Persona que ha cancelado su suscripción (0 = activo, 1 = cancelado)

Métricas clave:

Para poder determinar la efectividad de los modelos, se buscará usar las siguientes métricas:

- **Accuracy:** Este determina el porcentaje de usuarios que han sido clasificados correctamente.
- **Precisión:** Busca medir el porcentaje de predicciones positivas realizadas por el modelo corresponde realmente a usuarios que cancelaron su suscripción.
- **Recall de clase positiva:** Determina la efectividad en predecir a los usuarios que si han cancelado su suscripción.
- **F1 Score:** Este determina la combinación entre precisión y recall de la clase, por lo que permite evaluar las dos en conjunto.
- **AUC-ROC:** Busca determinar la efectividad del modelo para la separación de clases de manera binaria.
- **PR-AUC:** Esta métrica se utiliza cuando existe un desbalance de clases, mide el area bajo la curva de precisión y recall.

2.4 Análisis inicial de Dataset

La base de datos contiene 41,000 muestras, la cual cada una significa un usuario, los cuales incluyen los siguientes atributos:

Table 1: Diccionario de variables

Variable	Descripción	Tipo
Customer ID	Identificador de cada cliente	Númerica
Age	Edad del cliente	Númerica
Location	Estado de residencia del cliente	Categórica
Subscription Type	Tipo de suscripción	Categórica
Payment Plan	Forma de pago de suscripción: anual o mensual	Categórica
Num Subscription Pauses	Número de pausas de la suscripción (máx. 2)	Categórica
Payment Method	Forma de pago del usuario	Categórica
Customer Service Inquiries	Frecuencia de uso del servicio a cliente	Categórica
Sign up Date	Fecha en la que comenzó la suscripción	Fecha
Weekly Hours	Promedio mensual de horas que escucha música	Númerica
Average Session Length	Promedio de duración de cada sesión	Númerica
Song Skip Rate	Porcentaje de canciones que el usuario salta	Númerica
Weekly Played Songs	Promedio de canciones escuchadas por semana	Númerica
Weekly Unique Songs	Promedio de canciones únicas escuchadas por semana	Númerica
Num Favorite Artists	Número de artistas definidos como favoritos	Númerica
Num Platform Friends	Número de conexiones con otros usuarios	Númerica
Num Playlists Created	Número de playlists creadas	Númerica
Num Shared Playlists	Número de playlists compartidas públicamente	Númerica
Notifications Clicked	Número de notificaciones en las que hizo clic	Númerica
Churned	Indica si el usuario está activo o se dio de baja	Categórica de clase

3 Auditoría de Datos

Se identifican las variables con valores nulos o vacíos, los cuales al analizar el Dataset, no se encontraron ningunos.

También se realizó la identificación de errores en variables categóricas con el objetivo de encontrar errores ortográficos, gramaticales o de espacios que pudieran afectar los resultados. Se encontró un error en la variable "Location", en la cual se describía un estado como "Nebrasksa". Se ha sustituido a "Nebraska".

3.1 Codificación de Variables Categóricas

En las variables categóricas definidas en [1], se realiza una codificación a valores numéricos de tal manera que puedan ser incluidos como variables predictoras en los modelos. Este proceso se realiza con el método de One Hot Encoding, el cual busca convertir cada categoría en un resultado binario.

3.2 Reestructuración de Columnas

Se define el Index por la columna "customer_id" para tratar de darle mayor limpieza y estructura a la base de datos.

3.3 Estandarización de variables numéricas

Ya que existe diferencia en unidades para las variables numéricas, se ha determinado su estandarización. Esto se ha realizado con el método de Standard Scaler, el cual busca ajustar los valores de cada columna utilizando su media y desviación estándar. Con esto se logra tener las variables en una escala que permita que sean comparables entre ellas.

4 Análisis Exploratorio

4.1 Correlaciones

Para encontrar posibles relaciones entre variables numéricas, se ha utilizado una matriz de correlación que nos permita tener una observación inicial de si existen comportamientos proporcionales entre las variables.

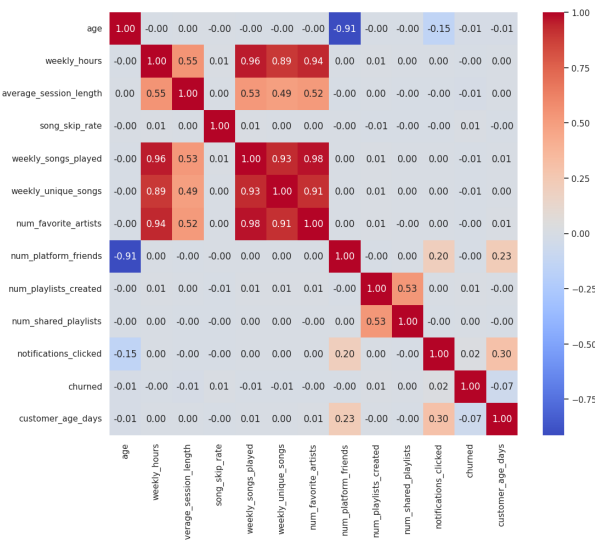


Figure 1: Matriz de Correlación entre Variables Numéricas.

Observando la matriz de correlación, se puede interpretar que existe una relación muy alta entre las columnas "weekly_hours", "weekly_songs", "weekly_unique_songs" y "num_favorite_artists". Esto tiene sentido debido a que al tener una mayor cantidad de música escuchada, aumenta la cantidad de canciones.

También se puede observar que la relación entre el número de artistas favoritos con el número de horas se podría interpretar como el grupo de usuarios que usan frecuentemente la aplicación y tienen una actividad alta en esta.

Analizando esta información y para reducir información que pueda ser redundante en el modelo, se busca eliminar las variables altamente correlacionadas y solo mantener el atributo de "weekly_hours".

4.2 Datos Relevantes

Analizando la columna predictora "churned", contiene 32,253 usuarios activos y 8,747 que han cancelado su suscripción. Esto en porcentaje se interpreta como 78.7% y 21.3% respectivamente.

5 Selección y Preparación de Modelo

5.1 Selección de Modelo

En el dataset, se busca predecir la variable "churned" que está clasificada de forma binaria. Al buscar un modelo de aprendizaje supervisado que permita predecir de forma correcta esta variable, se determinó el usar el modelo de **regresión logística**, el cual permite estimar la probabilidad de a qué clase pertenece el usuario en base a sus características y usos en la aplicación con el propósito de encontrar relaciones entre estos e identificar posibles usuarios que busquen cancelar su suscripción para así implementar nuevas estrategias que minimicen a los usuarios cancelados.

5.2 Funciones de Modelo

Para el modelo, se utiliza la función sigmoideal, la cual busca transformar el resultado de la combinación lineal de las variables independientes en un valor que está en un intervalo de 0 y 1. De esta forma, se puede interpretar como probabilidad de pertenecer entre clases. En este caso, se buscaría utilizar las características de la aplicación por usuario y predecir la pertenencia de cada uno.

Esta función tiene como entrada la combinación lineal de las variables y los pesos. Se define de la siguiente manera

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n. \quad (1)$$

Con el resultado de esta ecuación, se define la función sigmoideal como:

$$y = \frac{1}{1 + e^{-z}}. \quad (2)$$

El cual, como se menciona anteriormente, busca representar cada resultado como una probabilidad de pertenecer en la clase 1.

5.3 Selección de Datos

Una vez realizado el análisis exploratorio de datos y con la elección del modelo de regresión logística, se busca seleccionar las variables independientes que serán usadas para entrenar al modelo. Se define la columna "**churned**" como la variable a predecir (dependiente). Las variables independientes serán las siguientes:

- Age
- Location
- Subscription Type
- Payment Plan
- Average Session Length
- Num Subscription Pauses
- Payment Method
- Customer Service Inquiries
- Weekly Hours
- Num Platform Friends
- Num Playlists Created
- Num Shared Playlists
- Notificaciones Clicked

Cabe mencionar que las variables categóricas ya han sido transformadas de forma binaria a través de One Hot Encoding.

5.4 Split de Dataset

Tras agrupar las columnas para el entrenamiento del modelo, se busca dividir la información en tres conjuntos de datos: Test, Validation y Train. La parte de Test es la que contiene la mayor cantidad de información de los datos, el cual se usa para entrenar el modelo. Validation es el set de datos que se utiliza para probar correcciones y ajustar parámetros del modelo. En caso de que ya se haya aprobado las correcciones en Validation, se implementa Test para probar el modelo ante los últimos datos y observar su comportamiento en lo que sería su última prueba. En este caso, se ha utilizado un 70% del dataset en entrenar el modelo, un 10% en Validation y un 20% para test.

6 Entrenamiento del modelo

Antes de iniciar el entrenamiento, se ha incorporado un bias antes de cada observación, de tal manera que pueda formar parte de la combinación lineal que se utiliza en la función de hipótesis.

Sabiendo esto, el modelo comienza inicializando los pesos respecto a las variables dependientes. Para cada observación se calcula una combinación lineal entre sus características y sus pesos. Una vez hecho esto, se ingresa a la función sigmoideal para obtener su probabilidad de pertenencia hacia la clase 1.

Cuando termina de realizar esto, se comparan los resultados de la predicción respecto al valor real de la variable objetivo y la diferencia entre estos determina el error de la predicción.

En base a este error, se ajustan los pesos con el método de gradiente descendiente, el cual busca ajustar los pesos de tal manera que minimice el error del modelo lo más posible. En cada iteración busca calcular el gradiente de las observaciones y se actualizan con una tasa de aprendizaje (α) que ya es definida anteriormente.

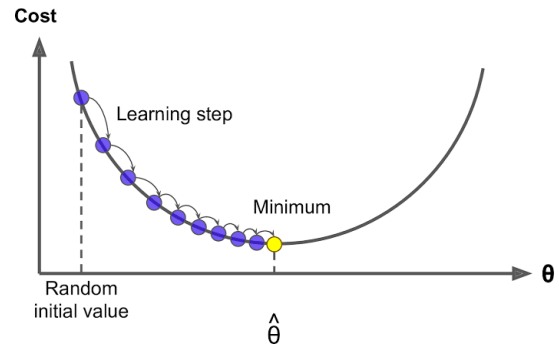


Figure 2: Representación visual del gradiente descendiente

Una vez implementada esta fórmula, los pesos se irán actualizando a través de las iteraciones de forma sucesiva en la dirección que haga que el costo disminuya.

Después de que se actualizan los pesos, se calculan nuevas probabilidades de cada muestra con los nuevos pesos y así se determina el costo del modelo utilizando la función de entropía cruzada (*Cross-Entropy*).

$$\text{cost} = -\frac{1}{M} \sum_{i=1}^M [y_i \log(h_0(x_i)) + (1 - y_i) \log(1 - h_0(x_i))] \quad (3)$$

Esta función de costo permite medir la distancia entre las probabilidades estimadas por el modelo respecto a las clases reales. El menor valor de costo representa un mejor ajuste entre el modelo y las clases reales.

Cuando termina la primera iteración, se almacena el valor de costo que ha arrojado la entropía cruzada. Esto se utiliza al final de entrenar el modelo para observar el comportamiento durante las iteraciones y como va mejorando mientras avanzan estos.

En el modelo se establece un máximo de 100 iteraciones. Este límite se utiliza para que el modelo no siga por iteraciones extremadamente largas ajustando los pesos buscando el óptimo. Puede llegar a tener mucho costo computacional o incluso ser redundante si en pocas iteraciones permite converger a una solución adecuada. El modelo puede terminar sus iteraciones de dos maneras.

- (1) El modelo llegó al límite de iteraciones.
- (2) El modelo alcanza una condición de convergencia definida.

Se acostumbra tener una mayor cantidad de iteraciones, pero al ser un modelo sin framework y que se considera "rústico", toma un tiempo considerable en realizar cada iteración.

7 Resultados

Para analizar los resultados, se incluye la gráfica de costo a través de cada iteración.

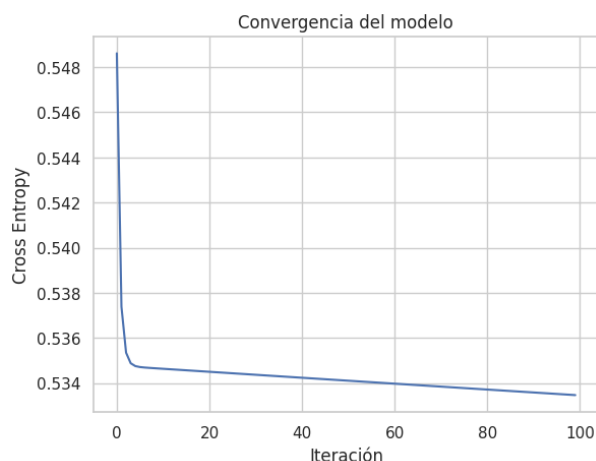


Figure 3: Convergencia del modelo a través de las iteraciones.

En el gráfico se observa que, en el transcurso de las iteraciones, se va disminuyendo el costo del modelo. Por lo tanto, si está funcionando de una manera adecuada el modelo. Cabe recordar que no es un modelo óptimo y que no contiene una cantidad grande de iteraciones.

Para las métricas, se ha utilizado Accuracy, Recall de la clase 1, matriz de confusión, AUC-ROC y PR-AUC. Accuracy nos permite saber qué tantas predicciones fueron correctas respecto a toda la base de predicción. Es decir, de todo el set de prueba, cuántos logró predecir de forma adecuada el modelo.

Recall sobre la clase 1 permite saber cuántos usuarios que realmente cancelaron se lograron detectar en base a las predicciones del modelo. Ambas métricas se obtienen a través de la matriz de confusión, la cual nos permite analizar la predicción de cada una de las clases. Esto lo hace a través de falsos y positivos. Captura cuáles sí fueron predichos de forma adecuada y cuáles fueron falsos positivos y falsos negativos.

En el caso de Area bajo la Curva (AUC), el cual es valor numérico el cual determina la efectividad del modelo, siendo 1 el valor de un modelo perfecto. También se implementa el Receiver Operating Characteristic (ROC), el cual es un gráfico que muestra el rendimiento del modelo para clasificar utilizando la tasa de verdaderos positivos contra los falsos positivos.

Como último, se utiliza la métrica de PR-ROC, el cual hace un balance entre precisión y recall para así medir la efectividad del modelo. Esta métrica tiene mayor efecto respecto a AUC cuando existe un desbalance de datos, por lo cual este será un valor importante en los resultados.

Estos se usarán para observar el comportamiento del modelo en general para clasificar ambas clases.

Los resultados del modelo son los siguientes:

Table 2: Resultados del modelo inicial

Métrica	Valor
Probabilidad mínima	0.1218
Probabilidad máxima	0.3968
Probabilidad promedio	0.2398
True Positive	0
True Negative	3205
False Positive	0
False Negative	895
Recall clase 1	0.0
Accuracy	0.7817
ROC-AUC	0.5213
PR-AUC	0.2459
Precision	0.0
F1	0.0

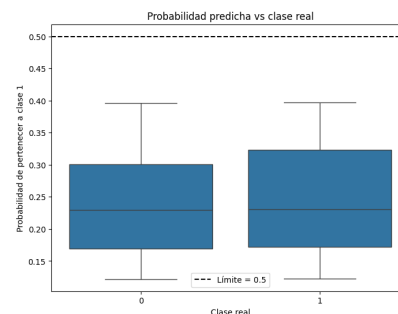


Figure 4: Caja de Bigotes de probabilidad por clase.

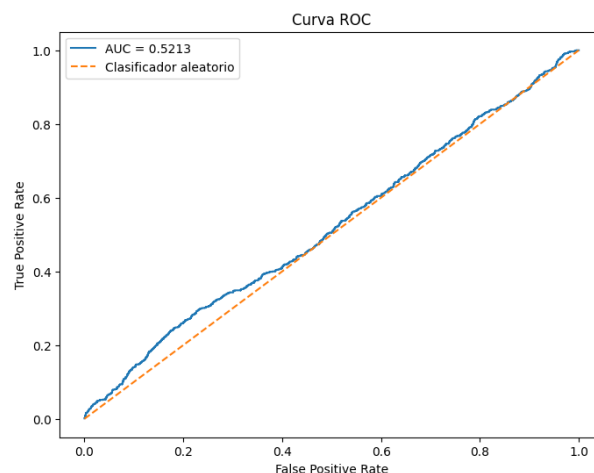


Figure 5: AUC-ROC de modelo

Analizando los resultados que ha arrojado el set de validación, se observa que el modelo tiene un accuracy de 78%, lo cual sería

adecuado. Pero, al analizar el recall de la clase 1, se observa que no ha clasificado a ninguno en esa clase.

Cabe mencionar que el umbral de decisión tras implementar la función sigmoideal es de 0.5. Es decir, si la probabilidad de churn es mayor a 0.5, se le asigna la clase 1. Observando la probabilidad máxima, el cual es casi un 40%, da a entender que el modelo predice que ningún usuario llega a cruzar ese umbral de 50%. Es por eso que el recall es de 0%, no existe ningún usuario que haya sido clasificado dentro de la clase 1. Esto también se puede observar en el gráfico de caja de bigotes, en el cual se observa que el modelo no ha encontrado una relación y diferencia significativa para clasificar entre ambas clases.

Es importante mencionar que un efecto importante que provoca esto es el desbalance de clases, a como se encontró en la exploración de datos, la clase de usuarios activos (0) tiene un 78% del dataset. Ahora, esto puede afectar el rendimiento del modelo y su forma de capturar a los usuarios que se buscan encontrar debido a este desbalance.

Otra métrica a observar es el gráfico [6], el cual indica la capacidad del modelo en predecir ambas clases. Al observar un valor AUC de prácticamente 0.5, se obtiene que existe casi la misma probabilidad de acertar si se escogiera aleatoriamente, lo cual indica que el modelo no está encontrando una forma adecuada de separar y clasificar estas clases. Es importante mencionar también que la métrica de PR-AUC es de 0.25%, lo cual indica que no se está teniendo una buena relación entre estas clases y en la clasificación adecuada de los datos. Investigando sobre esta métrica se obtiene que su línea base es cercana a la proporción de positivos en el dataset. Es decir, si tiene un valor cercano a 21% significa que es prácticamente igual a predecir de forma aleatoria.

Es importante entender esto y observar la importancia de tener múltiples métricas de evaluación para el modelo. Aunque el modelo muestra un accuracy relativamente alto debido al desbalance de clases, se muestra que no cumple lo que se busca ya que todos los predice como clase 0. Esto es lo que sesga a que haya un accuracy alto y el resto de las métricas determinan lo contrario.

7.1 Interpretación del modelo inicial

El modelo inicial al ser una regresión logística simple creada sin ningún framework, limitada por la cantidad de iteraciones y así encontrando poca diferencia entre las clases. Se buscaría implementar una regresión logística con mayor capacidad de iteraciones y con posibilidad de ajustar el desbalance de clases, modificando sus hiperparámetros de tal manera de encontrar una combinación que permita predecir de una mejor manera.

8 Implementación de Regresión Logística mejorada

Tras correr el modelo de forma manual e interpretar sus resultados, se buscaría observar el comportamiento y predicción al usar Scikit-Learn para realizar una Regresión Logística con mejoras de lo encontrado en el modelo anterior. Se buscará hacer un modelo con los siguientes ajustes:

- Se implementa una técnica de penalización hacia el error de la clase 1 para darle mayor importancia a esta clase.
- Se aumenta el número de iteraciones.

Para implementar en el framework, se buscaría hacer un split de datos que incluya un 70% de los datos en Train, un 10% de validation y un 30% de test. Se buscaría utilizar el comando de “stratify=y_train” ya que este permite que se tenga la misma proporción de clases entre el set de train y validation.

Una vez implementado el Scale de datos, se correría el nuevo modelo con los siguientes hiperparámetros:

- Solver = lbfgs
- max_iterations = 300
- Penalty = None
- class_weight = 'balanced'

El hiper parámetro Solver define el método de optimización del modelo, el cual es “Limited-memory Broyden–Fletcher–Goldfarb–Shanno”. Este es un optimizador de segundo orden que busca aproximar las derivadas de segundo orden utilizando las derivadas de pocas iteraciones anteriores. Es el optimizador por default en el framework de scikit learn. El número de iteraciones se ha limitado a 300 para que tenga mayor tiempo para converger hacia una solución y no se limite como el modelo que se realizó a mano. Penalty determina el método de regularización en el modelo, la cual busca reducir el sobre ajuste al penalizar modelos que son muy complejos. Este puede que sea utilizado posteriormente en caso de que lo requiera, pero por el momento no será utilizado. Como último, se implementa el comando “class weight”, el cual se utiliza para combatir el desbalance de clase. Esto lo realiza al automáticamente ajustar los pesos inversamente proporcionales a la frecuencia de clases. La fórmula que utiliza es la siguiente:

$$w_j = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot n_j} \quad (4)$$

En esta ecuación, w_j indica el peso para la clase j , n_{samples} es la cantidad de muestras que contiene la clase, n_{classes} indica el número de clases que existen (en este caso 2) y n_j indica el número de muestras para la clase j . De esta manera se le da un mayor peso a los errores que cometa prediciendo a la clase minoritaria, que es la que buscamos predecir. Cabe mencionar que existen métodos de sobre-muestreo como SMOTE, el cual busca crear muestras artificiales para aumentar el número de clases de tal manera que quede balanceado. Ahora, al tener solapamiento entre clases en el dataset, no se consideró adecuado hacer muestras sintéticas que podrían traer ejemplos ambiguos que estén en esa región de solapamiento. Es por eso que se determina ajustar el desbalance de clases al penalizar con mayor severidad el clasificar incorrectamente a los usuarios de la clase 1.

8.1 Resultados Modelo con Framework

Tras correr el modelo, se observan las métricas para así entender los resultados y poder determinar si es adecuado o si necesita ajustes. Los resultados son los siguientes:

8.1.1 Train.

- **Accuracy:** 0.7148%
- **Recall:** 0.4756%
- **Precisión:** 0.3692%
- **F1:** 0.4157%
- **PR-AUC:** 0.2875%

Matriz de confusión:**Table 3: Matriz de confusión para el conjunto de entrenamiento.**

	Predicción 0	Predicción 1
Real 0	17,602	4,975
Real 1	3,211	2,912

8.1.2 Validation.

- **Accuracy:** 0.7139%
- **Recall:** 0.4971%
- **Precisión:** 0.3724%
- **F1:** 0.4258%
- **PR-AUC:** 0.2925%

Matriz de confusión:**Table 4: Matriz de confusión para el conjunto de validación.**

	Predicción 0	Predicción 1
Real 0	2,492	733
Real 1	440	435

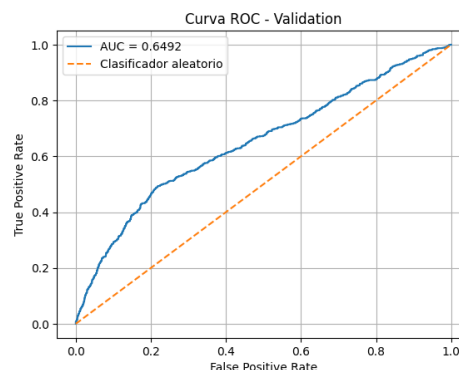
8.1.3 Test.

- **Accuracy:** 0.7098%
- **Recall:** 0.4671%
- **Precisión:** 0.3607%
- **F1:** 0.4071%
- **PR-AUC:** 0.2822%

Matriz de confusión:**Table 5: Matriz de confusión para el conjunto de prueba.**

	Predicción 0	Predicción 1
Real 0	5,003	1,448
Real 1	932	817

Se observa que, a comparación del modelo anterior, se disminuye el accuracy para el modelo, pero aumenta el valor de recall de la clase 1. Esto es significativo, ya que muestra que se ha ajustado de una manera positiva el desbalance de clase y aumenta significativamente la predicción de los usuarios en clase 1. Es importante observar el gráfico de AUC-ROC y la métrica PR-AUC para observar el comportamiento de clasificación entre clases del modelo.

**Figure 6: AUC-ROC de modelo mejorado**

Al obtener un valor AUC de 0.6492%, se observa que ya comienza a clasificar de una forma más práctica y que, aunque no es la mejor, se puede ver un mejor desempeño y separación de clases en este modelo. También analizando el valor de PR-AUC, se determina una mejoría de un 5% a comparación del modelo anterior.

Se observa que el modelo comienza a encontrar mejor relaciones entre las variables, permitiéndose ser más agresivo a comparación del modelo anterior gracias a la penalización de errores. De cualquier forma, se buscaría realizar un modelo con mayor complejidad y esperar que encuentre relaciones no lineales entre estas dos clases.

9 Implementación de XGBoost

Ahora, una vez que se ha implementado una regresión logística con ajuste de hiperparámetros para el desbalance de clases y aumento de iteraciones, se buscaría utilizar un modelo más robusto y no lineal que tenga mayor capacidad y flexibilidad de comprender patrones más complejos. Es por eso que se implementaría el modelo Extreme Gradient Boosting.

9.1 XGBoost

Este modelo de Machine Learning se basa en árboles de decisión, el cual tiene como método el crear un conjunto de árboles de decisión secuenciales y que cada árbol corrija el error cometido por el árbol anterior. Esto lo hace a través de la técnica de "boosting", la cual busca hacer una relación secuencial de modelos simples. Esta técnica inicializa un árbol con una predicción inicial, busca su error y en base a eso alimenta al segundo árbol para minimizar el error que cometió el árbol anterior. Cabe mencionar que este error se realiza usando el gradiente de una función de pérdida.

Para esto, se implementa la librería de **XGBoost** y siendo más específico el método de **XGBClassifier**. Este método usa por defecto como función de pérdida la función de pérdida entropía cruzada binaria, la cual busca minimizar la diferencia entre las probabilidades que ha dado el modelo y las clases reales.

Como parámetros a modificar, son los siguientes:

- **n_estimators:** Este parámetro establece el máximo de árboles de decisión que habrá de manera secuencial para mejorar la predicción sucesivamente.

- **learning_rate:** Este parámetro controla la contribución que da cada árbol al modelo final, su influencia.
- **max_depth:** Busca limitar la profundidad de cada árbol. Se usa para restringir la complejidad de los árboles y controlar un posible sobreajuste.
- **scale_pos_weight:** Este aumenta el peso asignado a la clase positiva en el entrenamiento. Este parámetro es el que permite balancear las clases, ya que es el que le da prioridad a la clase 1. Para este valor, se usó como medida práctica la división entre el porcentaje de la clase 0 y la clase 1. Es decir, $\frac{78.7}{21.3} \approx 3.72$.
- **eval_metric:** Esta variable es la que permite evaluar el desempeño del modelo basandose en una métrica en específico. En nuestro caso se utilizará la métrica *PR-AUC*, el cual muestra el desempeño del modelo en clasificar clases con desbalance.

Una vez mencionados estos parámetros, se alimenta la base de datos a distintos modelos para observar su desempeño.

9.2 Experimentación de modelos

Una vez mencionado los parámetros a usar y teniendo en mente los resultados del modelo anterior, se buscará ajustar los hiperparámetros de este modelo con el objetivo de mantener técnicas que puedan controlar el desbalance de clases, así como ajustes mínimos en los demás hiperparámetros buscando encontrar una combinación que aumente la métrica de PR-AUC y que sea equilibrado con Precisión y Recall.

Table 6: Experimentos realizados con XGBoost.

Modelo	<i>n_estimators</i>	<i>learning_rate</i>	<i>max_depth</i>	<i>scale_pos_weight</i>
XGBoost 1	300	0.05	4	–
XGBoost 2	300	0.05	5	3.72
XGBoost 3	300	0.03	3	3.72
XGBoost 4	1000	0.03	5	3.72

Table 7: Resultados obtenidos de experimentos con XGBoost.

Modelo	Precision	Recall	F1	ROC-AUC	PR-AUC
XGBoost 1	0.5120	0.0731	0.1280	0.6505	0.3603
XGBoost 2	0.3941	0.4571	0.4233	0.6508	0.3628
XGBoost 3	0.4025	0.4720	0.4345	0.6557	0.3779
XGBoost 4	0.3741	0.4583	0.4119	0.6492	0.3541

Analizando los hiperparámetros de los modelos y sus resultados, se puede determinar que el modelo óptimo para este problema ha sido el XGBoost 3. Este ha logrado mantener un buen valor de PR-AUC (0.3511) y de manera equilibrada un resultado adecuado para las demás métricas, especialmente para Recall.

9.3 Ajuste a Threshold

Ahora, una vez determinado este modelo, se busca evaluar entre distintos umbrales de decisión (threshold) de tal manera que tenga un balance adecuado entre precisión y recall. Para esto se realiza un análisis con un rango de 0.4 a 0.6, en el cual se analiza las métricas y matriz de confusión dependiendo de cual sea la frontera.

Table 8: Resultados obtenidos al evaluar diferentes umbrales de clasificación para XGBoost.

Threshold	Precision	Recall	F1	TP	FP	FN	TN
0.56	0.4362	0.4571	0.4464	400	517	475	2708
0.60	0.4421	0.4491	0.4456	393	496	482	2729
0.63	0.4461	0.4446	0.4453	389	483	486	2742
0.61	0.4438	0.4469	0.4453	391	490	484	2735
0.62	0.4447	0.4457	0.4452	390	487	485	2738
0.57	0.4358	0.4537	0.4446	397	514	478	2711
0.55	0.4315	0.4571	0.4440	400	527	475	2698
0.59	0.4376	0.4491	0.4433	393	505	482	2720
0.64	0.4466	0.4400	0.4433	385	477	490	2748
0.58	0.4354	0.4503	0.4427	394	511	481	2714
0.54	0.4275	0.4583	0.4424	401	537	474	2688
0.53	0.4210	0.4629	0.4409	405	557	470	2668
0.52	0.4148	0.4674	0.4395	409	577	466	2648
0.51	0.4100	0.4686	0.4373	410	590	465	2635
0.50	0.4025	0.4720	0.4345	413	613	462	2612
0.49	0.3954	0.4754	0.4318	416	636	459	2589
0.48	0.3881	0.4857	0.4315	425	670	450	2555
0.47	0.3820	0.4937	0.4307	432	699	443	2526
0.45	0.3622	0.5154	0.4255	451	794	424	2431
0.46	0.3686	0.5017	0.4250	439	752	436	2473
0.44	0.3480	0.5349	0.4216	468	877	407	2348
0.43	0.3248	0.5669	0.4130	496	1031	379	2194
0.42	0.3017	0.6217	0.4063	544	1259	331	1966
0.41	0.2691	0.6811	0.3858	596	1619	279	1606
0.40	0.2399	0.7577	0.3644	663	2101	212	1124

Estos resultados muestran el comportamiento que se espera. Cuando se reduce el umbral, aumenta la métrica de recall y disminuyen los falsos negativos, gracias a que una cantidad mayor de usuarios está siendo clasificada como clase 1. También como se vería esperar, al reducir este umbral aumenta los falsos positivos y una reducción a la precisión del modelo.

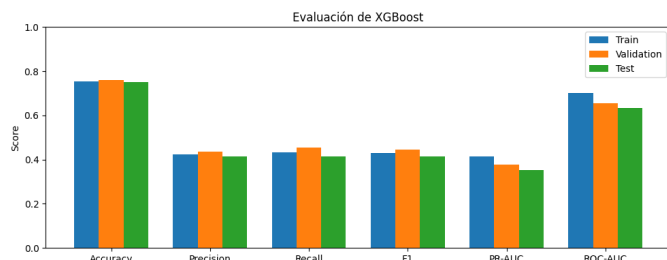
Ahora, analizando estos umbrales y acorde a lo que se busca en el modelo, lo cual es priorizar el F1 score, el cual es una combinación entre precisión y recall, se opta a ajustar el threshold a una probabilidad de 0.56. Esto es con el objetivo de evitar clasificar de manera equivocada a muchos usuarios que siguen siendo activos. En el caso de que se esté dispuesto a sacrificar una mayor cantidad, se podría considerar reducir este threshold a 0.42, lo cual de acuerdo a este set de validación, sería el punto donde mayor concentración podría haber de clasificar correctamente a los usuarios de clase 1.

9.4 Evaluación de Modelo en Test

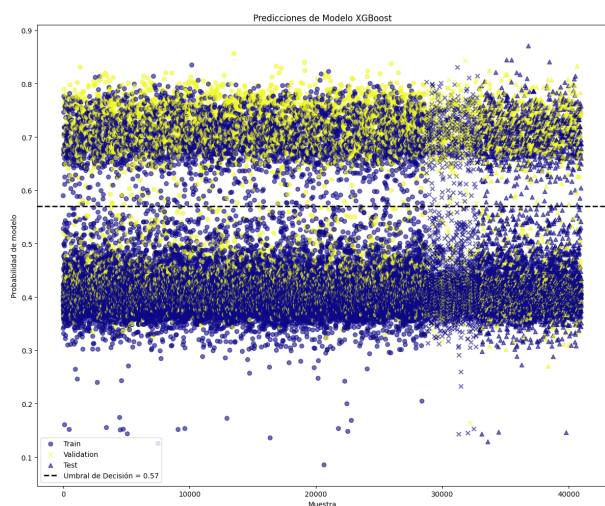
Una vez realizado el fine tuning y validado, se realiza la evaluación del modelo con el set de datos Test. Al ser implementado, se obtuvieron los siguientes resultados:

Table 9: Métricas para Test con XGBoost.

Métrica	Resultados
Accuracy	0.7513
Recall	0.4144
Precision	0.4165
F1	0.4149
PR-AUC	0.3553
ROC-AUC	0.6344

**Figure 7: Resultado de métricas para sets**

Tras evaluar este modelo con el set de test, se observa que las métricas tienen una disminución a comparación del set de validación. Aún así, el comportamiento general del modelo se mantiene estable, especialmente recall. El PR-AUC y el ROC-AUC también se disminuyen ligeramente, lo cual indica que si hay diferencia entre el desempeño que tuvo este modelo en validación y la prueba con datos que nunca ha visto.

**Figure 8: Predicción de probabilidad XGBoost**

Observando el gráfico anterior, el cual permite observar las predicciones realizadas en el modelo y los colores para distinguir las clases a través de los 3 sets, se puede observar que el modelo realiza una separación bastante amplia entre sus predicciones. Se puede observar en la parte superior del umbral abunda más los usuarios de la clase 1, aunque siga existiendo ruido entre ellas.

10 Conclusión

A partir del análisis realizado, se ha desarrollado y evaluado distintos modelos de clasificación que nos permiten identificar usuarios activos y usuarios que han cancelado su suscripción.

Table 10: Resultados de modelos.

Modelo	Precision	Recall	F1	ROC-AUC	PR-AUC
Regresión a Mano	0.0000	0.0000	0.0000	0.5213	0.2459
Regresión Scikit-Learn	0.3724	0.4971	0.4258	0.6492	0.2925
XGBoost	0.4358	0.4537	0.4446	0.6557	0.3779

Tras analizar los resultados de distintos modelos, se muestra que este dataset tiene una dificultad en separar completamente ambas clases, debido a la presencia de usuarios con comportamientos parecidos entre usuarios que decidieron cancelar su suscripción. Aún así, se pudo mejorar considerablemente el modelo a comparación de nuestro modelo inicial y se obtuvo un clasificador que permite dar una predicción relevante hacia los usuarios que cancelan.

References

- [1] E. Raza, "Gradient Descent," *Medium*, Sep. 5, 2019. [Online]. Available: <https://medium.com/@raza.shan83/gradient-descent-c568801d0b62>
- [2] Scikit-learn developers, "LogisticRegression," *Scikit-learn Documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- [3] Novus AFK, "Understanding Logistic Regression: A Beginner's Guide," *Medium*. [Online]. Available: https://medium.com/@novus_afk/understanding-logistic-regression-a-beginners-guide-73f148866910
- [4] GeeksforGeeks, "How Does the class_weight Parameter in Scikit-Learn Work?," *GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/how-does-the-classweight-parameter-in-scikit-learn-work/>
- [5] J. Brownlee, "SMOTE for Imbalanced Classification with Python," *Machine Learning Mastery*. [Online]. Available: <https://machinelearningmastery.com/sMOTE-oversampling-for-imbalanced-classification/>
- [6] Scikit-learn developers, "f1_score," *Scikit-learn Documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html
- [7] Google for Developers, "Classification: ROC and AUC," *Machine Learning Crash Course*. [Online]. Available: <https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc>
- [8] IBM, "¿Qué es XGBoost?," *IBM Think*. [Online]. Available: <https://www.ibm.com/mx-es/think/topics/xgboost>
- [9] A. R., "Scikit-learn solvers explained," *Medium*. [Online]. Available: <https://medium.com/@arnavr/scikit-learn-solvers-explained-780a17bc322d>
- [10] D. Yadav, "Weighted Logistic Regression for Imbalanced Dataset," *Medium*. [Online]. Available: <https://medium.com/data-science/weighted-logistic-regression-for-imbalanced-dataset-9a5cd88e68b>